数据结构期中复习题(by zxj)

第一章 绪论

一

二

第二章 线性表

一

第三章 栈和队列

一

第四章 串

一

第五章 数组和广义表

一

二

三

第六章 树和二叉树

一

二

为帮助学弟学妹复习期中考试所写，挑了一些题，不算全面，希望有所帮助

若发现错误，请联系作者赵薪迦，QQ：3040231533

宣传一下我们的数据结构朋辈导师答疑QQ群：945396021

第一章 绪论

一

1. 设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是_____。

```
x=0;
while (n>=(x+1)*(x+1))
x=x+1;
```

答案：

$O(n^{1/2})$

假设第 k 次循环终止，则第 k 次执行时，$(x+1)^2>n$，x 的初始值为 0，第 k 次判断时，$x=k-1$，即 $k^2>n$，$k>\sqrt{n}$，因此该程序段的时间复杂度为 $O(\sqrt{n})$。~~因此选 B。~~

二

求以下代码的时间复杂度

```c
int sum=0;
for(int i=1;i<n;i*=2)
{
    for(int j=0;j<i;j++)
        sum++;
}
```

答案：

王道书版本：

当外层循环的变量 i 取不同值时，内层循环就执行多少次，因此总循环次数为 i 的所有取值之和。假设外层循环共执行 k 次，当 $i=1,2,4,8,\cdots,2^{k-1}(2^{k-1}<n\leqslant 2^k)$ 时，内层循环执行 i 次，因此总循环次数 $T=1+2+4+8+\cdots+2^{k-1}=2^k-1$，即 $n<T<2n$，时间复杂度为 $O(n)$。

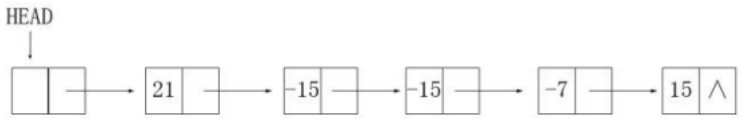
学长另解：

- 求时间复杂度，即求 `sum++` 执行次数和 `n` 之间的关系
- 设次数为S，写出几项并观察规律
  - S = 1+2+4+8+...+x
  - 其中x为不大于n的，2的最大整数次幂
- 可以确定项数约为$\log_2 n$，（估算即可，假定是个整数）
- 由等比数列求和公式
  - S=1*（2^$\log_2 n$ – 1）/ （2–1) = n – 1
- 所以时间复杂度为O(n)

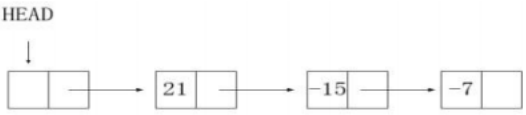
第二章 线性表

一

41.（15 分）用单链表保存 m 个整数，结点的结构为：[data][link]，且|data|≤n（n 为正整数）。现要求设计一个时间复杂度尽可能高效的算法，对于链表中 data 的绝对值相等的结点，仅保留第一次出现的结点而删除其余绝对值相等的结点。例如，若给定的单链表 head 如下：



则删除结点后的 head 为：



要求：

1）给出算法的基本设计思想。

2）使用 C 或 C++语言，给出单链表结点的数据类型定义。

3）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

4）说明你所设计算法的时间复杂度和空间复杂度。

答案：

(1)算法的基本设计思想

算法的核心思想是用空间换时间。

使用辅助数组记录链表中已出现的数值，从而只需对链表进行一趟扫描。

因为|data|≤n，故辅助数组 q 的大小为 n+1，各元素的初值均为 0。

依次扫描链表中的各结点，同时检查 q[|data|]的值，如果为 0，则保留该结点，并q[|data|]=1；否则，将该结点从链表中删除。

(2)

```C++
typedef struct node
{
    int data;
    struct node *link;
}NODE;
Typedef NODE *PNODE;
```

(3)

```C++
void func (PNODE h,int n)
{
    PNODE p=h,r;
    int *q,m;
    q=(int *)malloc(sizeof(int)*(n+1)); //申请 n+1 个位置的辅助空间
    for(int i=0;i<n+1;i++) //数组元素初值置 0
        *(q+i)=0;
    while(p->link!=NULL)
    {
        m=p->link->data>0? p->link->data:-p->link->data;
        if(*(q+m)==0) //判断该结点的 data 是否已出现过
        {
            *(q+m)=1; //首次出现
            p=p->link; //保留
        }
        else //重复出现
        {
            //删除
            r=p->link;
            p->link=r->link
            free(r);
        }
    }
    free(q);
}
```

(4)算法的时间复杂度为 O(m)，空间复杂度为 O(n)

第三章 栈和队列

一

5. 用栈实现将中缀表达式 8–(3+5)*(5–6/2)转换成后缀表达式，画出栈的变化过程图。

答案：

5．解答

中缀表达式转换成后缀表达式的具体步骤见 22 题，8-(3+5)*(5-6/2)转换成后缀表达式的具体过程如表 3.5 所示。

表 3.5　　中缀表达式 8-(3+5)*(5-6/2)转换成后缀表达式的具体过程

步　骤	OPTR 栈	字符串 Postfix	读入字符	主要操作
1	#		8-(3+5)*(5-6/2)#	'8' 加入字符串 Postfix
2	#	8	-(3+5)*(5-6/2)#	Push（OPTR，'-'）
3	#-	8	(3+5)*(5-6/2)#	Push（OPTR，'('）
4	#-(	8	3+5)*(5-6/2)#	'3' 加入字符串 Postfix
5	#-(	83	+5)*(5-6/2)#	Push（OPTR，'+'）
6	#-(+	83	5)*(5-6/2)#	'5' 加入字符串 Postfix
7	#-(+	835	)*(5-6/2)#	Pop（OPTR，'+'） '+' 加入字符串 Postfix
8	#-(	835+	)*(5-6/2)#	Pop(OPTR){消去一对括号
9	#-	835+	*(5-6/2)#	Push（OPTR，'*'）
10	#-*	835+	(5-6/2)#	Push（OPTR，'('）
11	#-*(	835+	5-6/2)#	'5' 加入字符串 Postfix
12	#-*(	835+5	-6/2)#	Push（OPTR，'-'）
13	#-*(-	835+5	6/2)#	'6' 加入字符串 Postfix
14	#-*(-	835+56	/2)#	Push（OPTR，'/'）
15	#-*(-/	835+56	2)#	'2' 加入字符串 Postfix
16	#-*(-/	835+562	)#	Pop（OPTR，'/'） '/' 加入字符串 Postfix
17	#-*(-	835+562/	)#	Pop（OPTR，'-'） '-' 加入字符串 Postfix
18	#-*(	835+562/-	)#	Pop(OPTR){消去一对括号
19	#-*	835+562/-	#	Pop（OPTR，'*'） '*' 加入字符串 Postfix
20	#-	835+562/-*	#	Pop（OPTR，'-'） '-' 加入字符串 Postfix
21	#	835+562/-*-	#	return Postfix

学长给出的步骤：

- 初始化一个栈
  - 用于保存暂时还不能确定运算顺序的**运算符**
- 从左到右处理各个元素，直到末尾。可能遇到三种情况：
  - 遇到操作数
    - 直接加入后缀表达式
  - 遇到界限符

        - 遇到 `(`
            - 直接入栈
        - 遇到 `)`
            - 依次弹出栈内运算符并加入后缀表达式
            - 直到弹出 `(` 为止
            - 注意： `(` 不加入后缀表达式
    - 遇到运算符
        - 依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式
        - 若碰到 `(` 或栈空则停止
        - 再把当前运算符入栈
- 按上述方法处理完所有字符后
    - 将栈中剩余运算符依次弹出，并加入后缀表达式

第四章 串

一

求 `T = 'abaabc'` 的next数组和改进后的nextval数组

答案：

> 注意：
>
> next数组可能存在不同版本，这里给出的是学校考试和王道参考书版本
>
> 和ACwing上KMP算法next数组不同
>
> ACwing上严格来说是部分匹配值表
>
> 而此版本是直接跳转比较的位置

序号	1	2	3	4	5	6
字符	a	b	a	a	b	c
next	0	1	1	2	2	3
nextval	0	1	0	2	1	3

这里学长给出一种方法：

- 给出几个概念
  - 非平凡前缀：指除了最后一个字符以外，一个字符串的全部头部组合
  - 非平凡后缀：指除了第一个字符以外，一个字符串的全部尾部组合
  - 部分匹配值：非平凡前缀和非平凡后缀的最长共有元素的长度
- 先求部分匹配值表：

序号	1	2	3	4	5	6
字符	a	b	a	a	b	c
最长公共前后缀	/	/	a	a	ab	/
部分匹配值	0	0	1	1	2	0

- 再将部分匹配值加一，并后移一位，得到next数组，特殊地，next[1]=0

序号	1	2	3	4	5	6
字符	a	b	a	a	b	c
部分匹配值	0	0	1	1	2	0
next	0	1	1	2	2	3

- 最后根据以下过程，由next数组求出nextval数组

C++

```
nextval[1]=0;
for (int j=2; j<=T.length; j++)
{
    if(T.ch[next[j]]==T.ch[j])
        nextval[j]=nextval[next[j]];
    else
        nextval[j]=next[j];
}
```

序号	1	2	3	4	5	6
字符	a	b	a	a	b	c
next	0	1	1	2	2	3
T.ch[next[j]]	/	a	a	b	b	a
nextval	0	1	0	2	1	3

第五章 数组和广义表

一

数组A[1...8，-2...6，0...6]以行为主序存储，设第一个元素的首地址是78，每个元素的长度为4，试求元素A[4,2,3]的存储首地址。

答案：958

- 三维数组以行为主序存储，其元素地址公式为：
  - LOC (A_{ijk}) = Loc(A_{c1c2c3}) + [($i-c_1$) *V_2*V_3+ ($j-c_2$)*V_3 + ($k-c_3$)]*L
  - c_i，d_i 是各维的下界和上界
  - V_i= d_i – c_i+1是各维元素个数
  - L是一个元素所占的存储单元数
- 则LOC ($A_{4,2,3}$)=78 + [(4-1)*9*7+(2-(-2))*7+(3-0)]*4=958

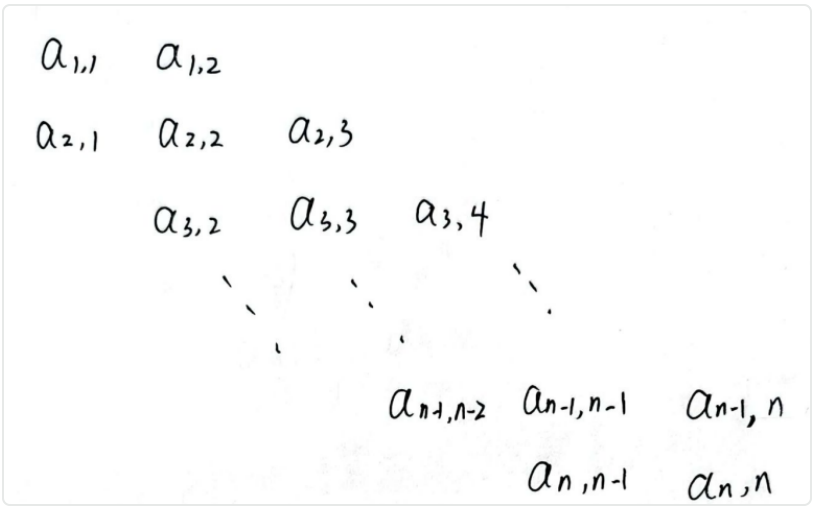
二

6. 三对角矩阵$(a_{ij})_{n\times n}$是除主对角线以及与主对角线相邻的两条对角线以外的元素全为0，将其3条对角线上的元素逐行地存于数组$B[1..3n-2]$中使得$B[k]=a_{ij}$，求：

(1) 用i、j表示k的下标变换公式。

(2) 用k表示i、j的下标变换公式。

答案：



(1)

对于$a_{i,j}$

①当$a_{i,j}$不在第一行

$a_{i,j}$自身和之前元素总个数：

$$2 + (i-1)\times 3 + (j-i) = 2i + j - 2$$

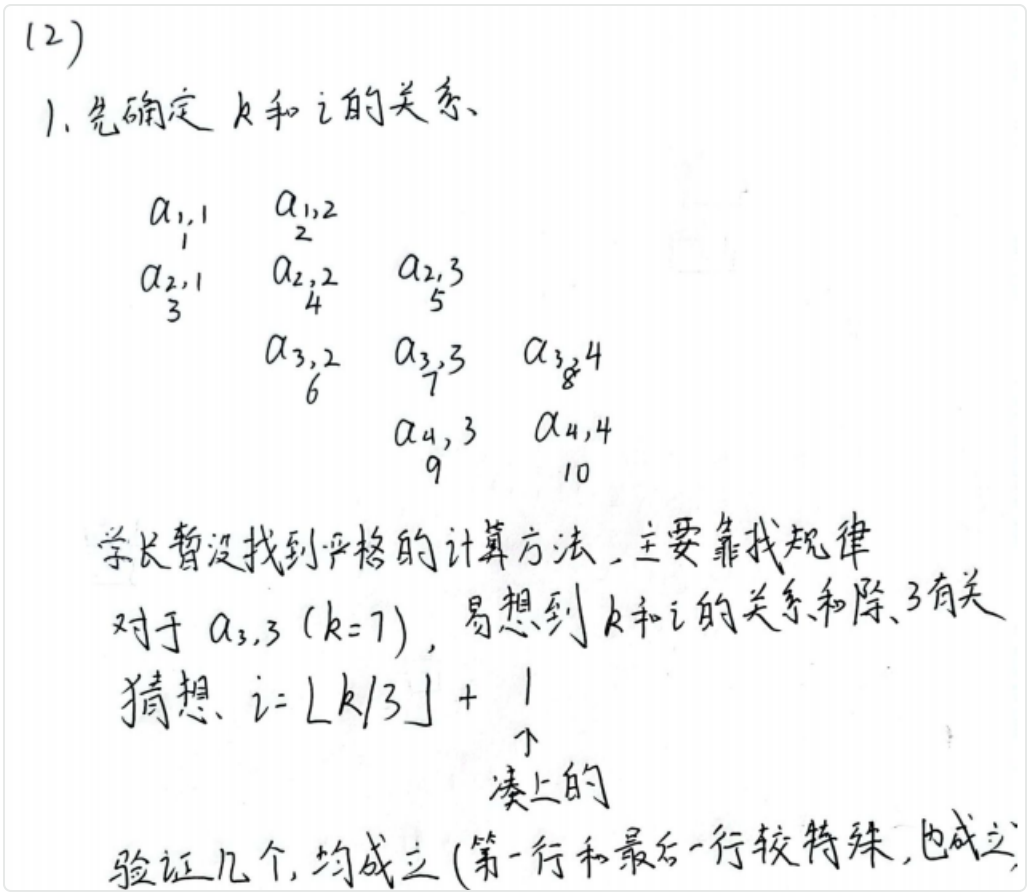
↑ 第1行　↑ 第2n i-1行　↑ 第i行

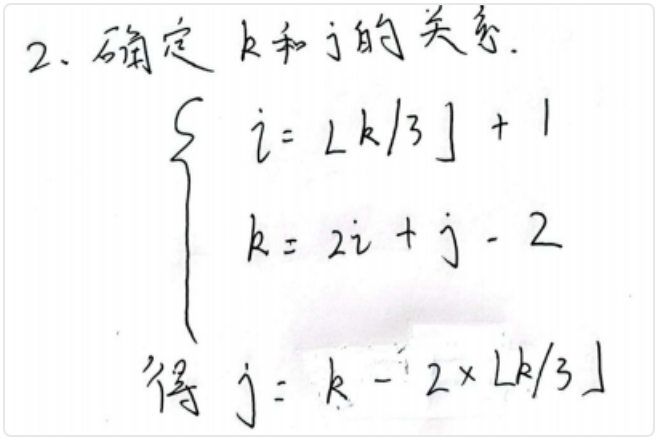
注意到数组从1开始，则 $k = 2i + j - 2$

②当$a_{i,j}$在第一行

上式包含之

综上，$k = 2i + j - 2$





注意：若题目i,j,k的起点改变，答案也会改变

> 若有同学找到第二问严格计算方法，请教教学长，感激不尽！

三

利用广义表的Head和Tail运算，把原子d分别从下列广义表中分离出来

```C++
L1=(((((a),b),d),e))
```

```
1   L2=(a,(b,((d)),e))
```

答案：

Head(Tail(Head(Head(L1))))

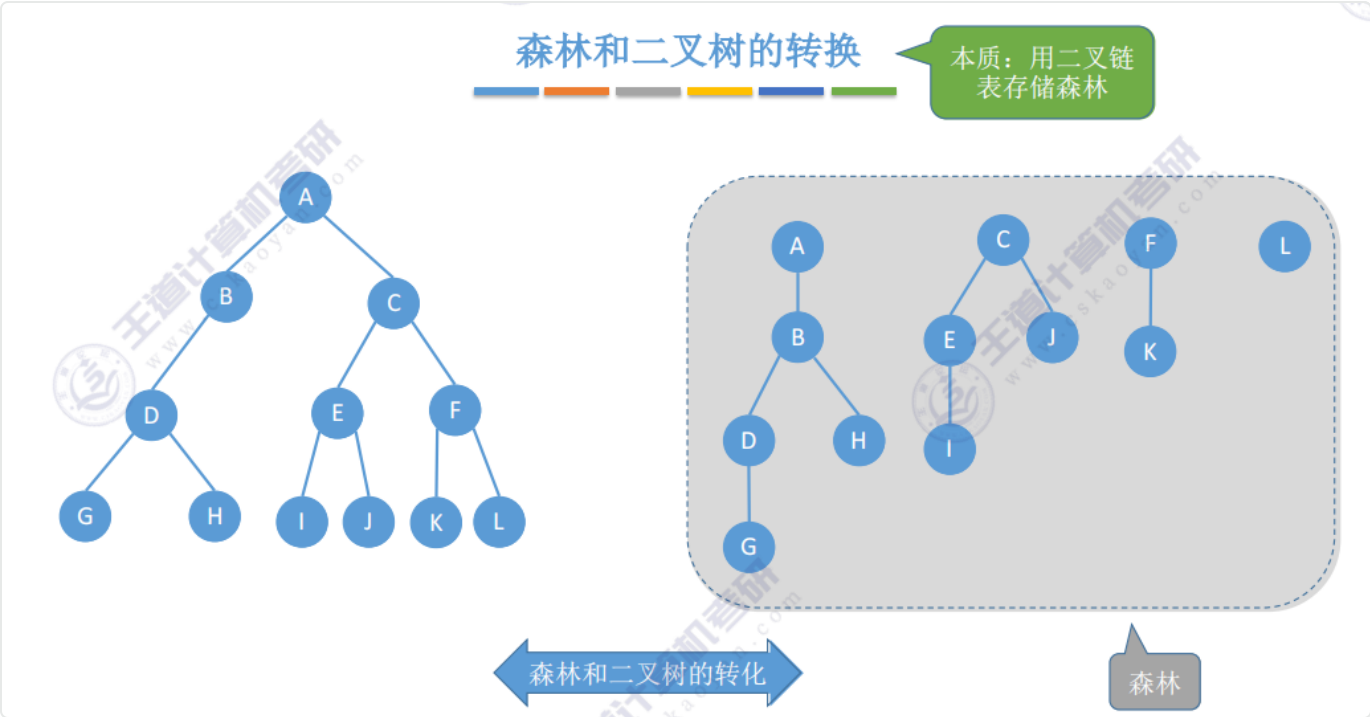
Head(Head(Head(Tail(Head(Tail(L2))))))

注意：

- 表头是表中第一个元素，它可以是原子，也可以是子表
- 表尾必定是子表

第六章 树和二叉树

一



- 森林的左孩子右兄弟表示法
- 森林中各个树的根节点视为兄弟关系

二

> 二叉树的递归算法比较常考

二叉树的带权路径长度(WPL)是二叉树中所有叶结点的带权路径长度之和，也就是每个叶结点的深度与权值之积的总和。

给定一棵二叉树 T，请你计算并输出它的 WPL。

注意，根节点的深度为 0。

样例

```
输入：二叉树[8, 12, 2, null, null, 6, 4, null, null, null, null]如下图所示：
    8
   / \
  12  2
     / \
    6   4

输出：32
```

C++

```cpp
struct TreeNode
{
    int val;
    TreeNode *left;
    TreeNode *right;
};
```

思路：

- 由于我们需要遍历所有的叶结点
  - 也即我们要遍历树中所有结点
  - 所以可以利用**深度优先搜索**
- 由于最终的长度包含**结点深度**
  - 所以深度优先搜索的参数中需要包含
    - 当前**树结点** `root`
    - **深度** `depth`

步骤：

- 每次首先判断当前树结点是否为空
  - 若为空则直接返回 `0`
- 其次判断是否为叶节点
  - 若为叶节点则返回**该结点值与深度的乘积**
- 否则
  - 继续**搜索其左子结点及右子结点**
- **时间复杂度**
  - O(n)

代码：

C++

```cpp
int sum(TreeNode* root,int level)
{
    if(!root) return 0;
    if(!root->left&&!root->right)
    {
        return root->val*level;
    }
    return sum(root->left,level+1)+sum(root->right,level+1);
}

int pathSum(TreeNode*root)
{
    return sum(root,0);
}
```